Future Generation Computer Systems 155 (2024) 179–192

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

A reference architecture for serverless big data processing☆

Sebastian Werner *, Stefan Tai

ISE, TU Berlin, Einsteinufer 17, Berlin, 10587, Germany

ARTICLE INFO

Keywords:
Serverless data processing
Application platform co-design
Serverless reference architecture
Function as a Service
Software engineering
Cloud computing

ABSTRACT

Despite significant advances in data management systems in recent decades, the processing of big data at scale remains very challenging. While cloud computing has been well-accepted as a solution to address scalability needs, cloud configuration and operation complexity persist and often present themselves as entry barriers, especially for novice data analysts. Serverless computing and Function-as-a-Service (FaaS) platforms have been suggested to reduce such entry barriers by shifting configuration and operational responsibilities from the application developer to the FaaS platform provider. Naturally, "serverless data processing (SDP)", that is, using FaaS for (big) data processing, has received increasing interest in recent years.

However, FaaS platforms were never intended to support large data processing tasks primarily. SDP, therefore, manifests itself through workarounds and adaptations on the application level, addressing various quirks and limitations of the FaaS platforms in use for data processing needs. This, in turn, creates tensions between the platforms and the applications using them, again encouraging the constant (re-)design of both. Consequently, we present lessons learned from a series of application and platform re-designs that address these tensions, leading to the development of an SDP reference architecture and a platform instantiation and implementation thereof called CREW. Mitigating the tensions through the process of application platform co-design proves to reduce both entry barriers and costs significantly. In some experiments, CREW outperforms traditional, non-SDP big data processing frameworks by factors.

1. Introduction

Continuously and with ever-increasing speed, applications are being built on top of software platforms. The software platforms correspondingly must support many use cases, which creates a wide range of both features but also quirks and platform-specific constraints driving application architecture and changes. Application demands, in turn, force developers to seek platforms with minimal constraints and the features best suited to address their individual needs, thus incentivizing platforms to adapt to changing demands. Alternatively, changes in application demands may suggest the development of a new (variant of a) software platform better suited to address application needs with minimal adaptation costs for applications. Managing the duality of addressing these demands is the goal of application platform co-design [1] – an approach to address the design tensions in applications driving new platform designs and platforms driving new application designs.

One area of particular interest are applications that must process large quantities of data. Over the last decades, it has become increasingly simple to generate and collect data [2], be it from extensive physics experiments, from IoT, or from logging and monitoring large distributed systems. While business and research collect data at an extraordinary pace, systems to analyze this data, however, must scale to handle the increasing volume, variety, and velocity of the data [3–5]. Consequently, industry actors and researchers must continually (re-) design and expand systems to store, evaluate, and process big data at an increasing scale.

Notably, developers can choose between a magnitude of different environments and components to build their data processing systems for each application domain. However, these choices usually also come with varied configuration, management and usage complexity, creating increasingly high entry barriers for the average user. Contemporary approaches to bridge this complexity gap use cloud platforms, reducing the choice to either fully managed or partially self-managed data analytics platforms. For example, a user can elect to run Apache Spark [6,7] in a fully managed manner, e.g., using Amazon EMR [8], and thereby relies on cloud providers for creating and owning large-scale data processing clusters. However, "distributed computing remains inaccessible to a large number of users" [9], as many tuning, sizing, programming, and operating challenges remain.

One promising trend in this direction is the use of serverless platforms, e.g., Function-as-a-Service (FaaS) platforms to run large-scale

☆ This document is in part published as Werner (2023).

* Corresponding author.
E-mail addresses: sw@ise.tu-berlin.de (S. Werner), st@ise.tu-berlin.de (S. Tai).

https://doi.org/10.1016/j.future.2024.01.029
Received 20 July 2023; Received in revised form 26 January 2024; Accepted 27 January 2024
Available online 30 January 2024

data processing [10], reducing barriers to entry by sharply separating operational concerns between development matters and those related to usage [9,11–13]. Here, all scaling and infrastructure decisions are made by the cloud provider. Applications built on such a platform can quickly respond to workload changes and scale even down to zero if no work arrives. Thus, industry actors and researchers have started to utilize FaaS to process data, as "ultimate elasticity and operational simplicity [...] in the context of data analytics sounds appealing" [14]. Data processing approaches that utilize FaaS obviate management and tuning of clusters and reduce the need for resource-based sizing and scaling decisions [10,15,16].

We argue that existing entry barriers, e.g., high configuration complexity and cost of ownership, are addressable FaaS approaches [17,18]. At the same time, serverless solutions can outperform traditional big data processing systems in terms of cost and performance for exploratory data analysis [19]. Still, while generic FaaS platforms offer a simplified computing model, they do not specifically address data processing needs [12–14,20], thereby creating design tensions between data processing applications and serverless platforms to address these needs. Consequently, in this paper, we ask the question: **How can we co-design serverless data processing applications and platforms to reduce entry barriers and ease existing design tensions?**

Toward that end, we present the following contributions:

- A reference architecture for implementing serverless data processing systems
- `CREW` – a new serverless data processing framework and serverless platform that addresses tensions using application platform co-design
- A demonstration and evaluation of `CREW`

Note, while we present `CREW` as a new serverless data processing framework and platform, it is not intended as a production-ready system. Instead, we use `CREW` to demonstrate the feasibility of the application platform co-design approach to address design tensions we observed through our reference architecture.

The remainder of this paper is structured as follows: First, in Section 2, we present the serverless data processing reference architecture and currently manifested tensions. Moreover, in Section 3, we introduce `CREW`, a new serverless data processing framework and platform that addresses these tensions. In Section 4, we evaluate `CREW` compared to Apache Spark. Lastly, we discuss related work in Section 5 and conclude this work.

2. Serverless big data processing

In the following, we aim to identify "a family of similar systems and to standardize nomenclature"[21] for serverless data processing (SDP) using a literature study (see Appendix A). Moreover, we present common tensions found in existing serverless data processing architectures, propose a reference architecture for SDP and pinpoint the interactions within the reference architecture that are common sources of these tensions, providing an outlook on how to address these tensions in the next generation of serverless data processing systems.

2.1. Common serverless data processing systems

Serverless data processing typically involves multiple, often interchangeable, platforms and services to address different functional and non-functional goals. Besides Function-as-a-Service (FaaS) offerings, fully managed storage services and fully managed orchestrators such as AWS StepFunctions are common. The main benefit of using serverless platforms instead of analytics engines with a traditional deployment model such as Apache Flink lies in the highly elastic, event-driven resource provisioning with minimal operational overhead offered through the serverless model. Consequently, any data processing task that benefits from ad-hoc computation, as well as, distributed batch data processing [9,12–14,22–25] shows to benefit significantly.

Table 1
Overview of serverless data processing frameworks extracted through an SLR conducted between 2018–2022 (see Appendix A). Note that some of the frameworks are not publicly available or are no longer maintained.

Name	Release Year	Last Activity	Driver	Computing	Storage
PyWren [9]	2016	2018	✓	✓	
gg [12]	2017	2022	✓	✓	✓
Flint [22]	N.A.	N.A.	✓	✓	
Lambada [14]	N.A.	N.A.	✓	✓	✓
Starling [26]	N.A.	N.A.	✓	✓	✓
CRUCIAL [20]	2019	2021	✓	✓	✓
Locus [23]	2016	N.A.			✓
Qubole [27]	2018	2019	✓	✓	✓
Opvis [28]	N.A.	N.A.	✓		✓
MARLA [29]	2017	2021	✓	✓	✓
Ooso [30]	2017	2017	✓	✓	✓
Wukong [24]	2019	**2023**	✓	✓	✓
Berkley ML [31]	N.A.	N.A.	✓		✓
SCAR [32]	2017	N.A.	✓	✓	✓
CCoDaMiC [33]	N.A.	N.A.	✓		✓
Corral [34]	2018	2021	✓	✓	✓
Lithops [35]	2018	**2023**	✓	✓	✓
`CREW`	2020	**2023**	✓	✓	✓

However, this creates a large number of "moving parts" that each require attentive configuration and management, which limits the ease of use and may create different entry barriers [9]. Hence, programming frameworks and tools have emerged to automate the generation, deployment and orchestration of serverless data processing applications. In Table 1, we present an overview of the most predominant frameworks discovered in a systematic literature review conducted from 2018–2022, along with the advancements they introduced. These advances all represent workarounds or approaches to deal with the tensions between applications and platforms discussed in this section. Note that some of the frameworks are not publicly available or no longer maintained, while a small number are still in development.

2.2. A reference architecture

Based on a careful review of the architectures of the discovered frameworks (Table 1), we compiled a reference architecture for serverless data processing. Fig. 1 gives an initial high-level overview of the components, the relationships and the domains of responsibility of this reference architecture. It further highlights the different platforms (blue) that must interact with application components (gray), including interactions typically handled by the supporting processing framework (green). The components within this architecture are the most commonly modified parts of the original serverless model and thus represent the defining features of serverless data processing applications.

We separate serverless data processing applications into four logical planes. Firstly, on the **User Plane**, data analysts define a **processing job** using, for example, query languages [26], map-reduce APIs [35,36], or full-fledged Spark APIs [22]. Secondly, the **Control Plane** is responsible for translating, packaging, and deploying the serverless execution engine and managing the execution at runtime. Thirdly, the **Execution Plane** is responsible for realizing the analytics logic, usually packaged as multiple small functions or containers. Lastly, the **Data Plane** is responsible for storing and retrieving intermediate data and raw data for the processing job,[1] as well as for enabling the observability for the control plane.

[1] Usually in the form of managed storage service, such as S3.

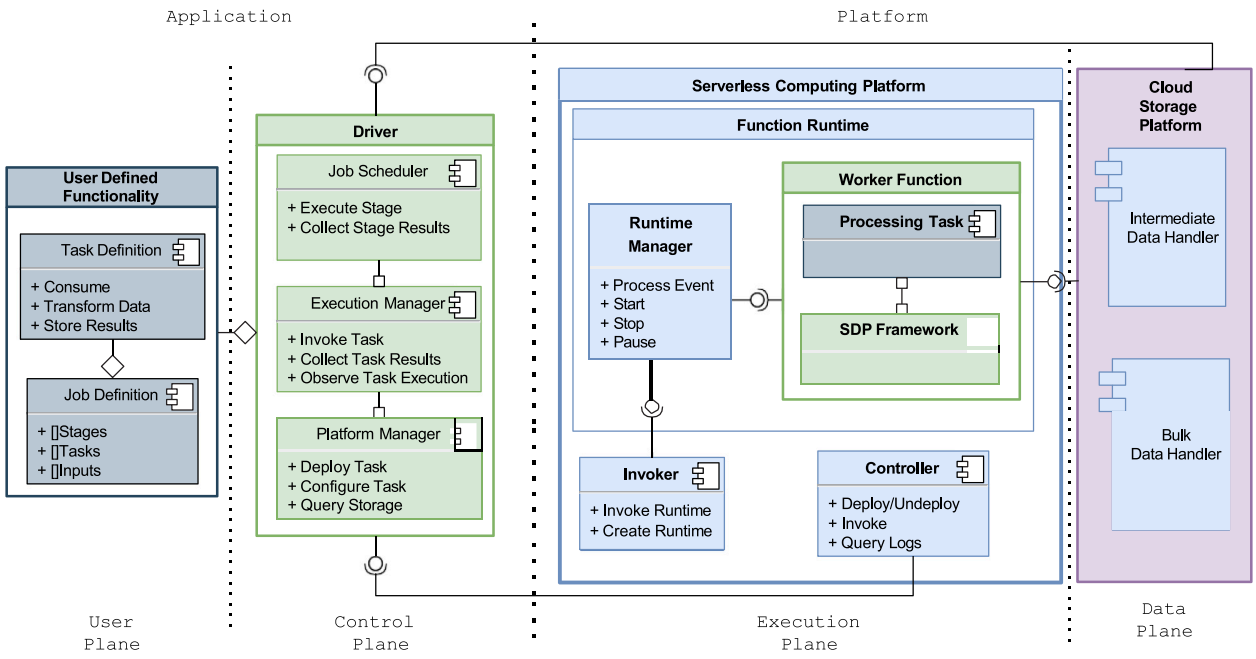

Fig. 1. Generalized serverless data processing application reference architecture, showing how the different platforms (blue) interact with the application (gray) and the supporting processing framework (green). Including both platform- and application-side components that can influence overall processing qualities.

In the following, we will briefly discuss these planes and how cross-cutting concerns within reveal tensions in the design of serverless data processing applications that framework developers must address.

User plane. Within the User Plane, the data analyst is defining all the functionality, e.g., the tasks and configuration that the serverless data processing framework will use to perform the analytics job. Naturally, the way in which these functions and configurations are created is limited by the framework's API and depends on the provided programming abstractions. Although platform details are often hidden from the data analyst, the underlying platform can still influence the means offered to define processing jobs. For example, writing code to run on AWS might need different considerations in terms of data sizes per function than writing code for OpenWhisk, as the limits of OpenWhisk can be tuned more than those for AWS. Moreover, it is up to the SDP framework to optimize and plan how the provided configurations and user defined functions (UDFs) are turned into a series of processing steps executed by serverless function runtimes.

Control plane. The control plane typically contains a job driver, equivalent to a driver in traditional processing systems, responsible for translating a user-defined job description into something that can be executed. Thus, these drivers embody the core functionality of an SDP framework, translating UDFs into concrete deployment packages (functions) and providing execution plan (events) to run the job.

Thus, the driver is strongly coupled to the **Worker-Function**, which receives the event and executes the corresponding task in accordance with the UDFs. This strong coupling also means that the driver has to observe event completion, error recovery, straggler mitigation and configuration optimizations.

Consequently, the driver is a central component responsible for managing the interaction between applications and platforms and, thus, a central point of tension:

T1 Tension from the mapping of application tasks (e.g., processing operations) to concrete deployment and configuration, as this is needed in a far more granular way than in other data processing systems. For example, platform-driven limitations on package sizes, runtimes, and memory can limit the possible functionality of provided code and, thus, the capability of processing tasks, which can lead to increased cold-starts [37] or increased overhead due as functions need to be split up.

T2 Tension from the coarse configuring. For example, a function that is configured with too little memory can lead to faults, execution issues, or increases in management overhead, while a function with too much memory can lead to unnecessary costs. At the same time, the needed memory is often not known in advance, as it depends on the input data generated by the previous stage [38,39].

T3 Tension from the scheduling and observation between the driver and execution platform. As, in many cases, the driver and the platform share responsibilities for scheduling the execution of the processing job, it is crucial to align both. Towards that end, the platform must provide consistent and timely feedback to the driver, and the driver should not overload the platform with status requests.
Here, many workarounds are used such as the *Job-Server and Workpulling-Functions* [24,28] which bypass the platforms resource optimizations, the *Function-Chaining* [14,29] which add more complex fault mitigation needs [40] or approaches such as bootstraping [13] which introduced additional cost and overhead.

Execution plane. The execution plane provides the processing power to perform each task that a driver generates. The goal of each serverless data processing application (SDPA) is to scale in response to the workload without the interaction of the data analyst. Naturally, the selected platform strongly influences the performance and cost [41] of an SDPA. Each platform presents unique limitations that a driver must work around [1], e.g., configuration-sensitivities [39], event generation support [1], observability [40] and acceleration [42].

Moreover, how worker functions are built also strongly influences system quality. For example, the use of pre-compiled binaries to execute specific operations leads to smaller deployment packages, which can reduce cold-starts [37,43].

The capabilities and limitations of the serverless execution platform are thus a central point of tension for SDPA. Among them, the following tensions are of particular importance:

T4 Tension from the atypical use of serverless systems for data processing, e.g., briefly running fleets of functions from a singular client.[2] Which can significantly increase cold-start panelties [37, 44].

T5 Tensions from the stateless programming model, which leads to the need to store intermediate results in a separate storage system. Thus, data must always move between the storage system and the serverless platform, even for small results [45]

Data plane. The data plane mainly consists of means to handle intermediate data (**T5**) and the bulk data that needs to be processed. For bulk data, object stores are most common [19]. Some approaches reuse the bulk data object store for intermediate results, while other approaches utilize fully managed message queues, in-memory databases, or purpose-built middleware for intermediate storage. However, all these approaches result in a large amount of data movement between the storage system and the serverless platform, leading to significant tensions:

T6 Tension from the quickly shifting data access patterns (e.g., dynamic partitioning, shuffling and merging of data, and bulk access, as well as, line-wise or byte-wise access). Limiting how specialized a storage system can be for serverless data processing [46].

T7 Tension from the highly parallel nature of serverless processing. An SDPA might spawn thousands of functions that access the same object, each reading only very small parts. Thus, the storage system must be able to distribute the load adequately and in a more fine-grained way than for other use cases such as Spark [19].

T8 Tension from heavy usage of metadata queries. Differently from other applications, SDPA heavily misuses metadata queries to orchestrate the processing, e.g., to see if a series of functions is finished. Thus, the storage system must provide performant and consistent metadata to ensure smooth operation. At the same time, SDPA should limit the number of needed queries.

For each of these tensions, we found manifold workarounds in the literature that aim to address platform limitations. However, we note that most of these workarounds solely address the symptoms of the tensions and not the underlying causes, namely missing serverless data processing features or storage systems features on the platform side. Features hopefully addressed in the next generation of serverless data processing.

2.3. Next-generation serverless data processing

The model of event-driven scalability and other desirable properties of serverless platforms have fueled the development of many serverless data processing frameworks (see Table 1). However, a careful review of these frameworks also reveals a fundamental issue with current serverless platforms, such as AWS Lambda or Google Cloud Functions, which is that these platforms were never designed with data processing in mind. Thus, some features and behaviors of these platforms create challenges in using their full potential.

Using the reference architecture (Fig. 1), we more closely identify these tensions within the boundaries between the application/platform and platform/platform. From our review of existing literature and source code, we see that so far, tensions have only been addressed through workarounds on the application level or by finding novel service compositions. However, we argue that these tensions must be addressed by both platforms and applications to realize this untapped potential. For example, bypassing the event system to invoke functions [24] to improve function utilization impacts the platform's ability to predict workloads. Hence, it may improve performance but may impact stability. However, if the platform would offer a feature such as used in [24], these stability issues could be addressed. This dichotomy of where and how tensions are addressed also explains why some tensions, i.e., T5/T7/T8, are not discussed at all by current applications, limiting the potential and current qualities of SDP.

Consequently, we argue that the next generation of serverless data processing systems must change both application architectures and serverless platforms (serverless compute platforms, storage systems and other composable services) to address these tensions. Showcasing the benefits of this application platform co-design [47] approach, we present CREW in Section 3. These re-designs also reveal potential future serverless platform features that could, with little impact on the current business model, be adapted by current serverless platforms. Moreover, we can demonstrate that these changes can lead to significant improvements in performance, cost, and reliability of serverless data processing applications and start to become competitive with traditional data processing systems like Apache Spark. However, we must point out that the concrete approach on how to best select appropriate re-designs is a complex engineering task [1,47].

3. CREW – a next-generation serverless data processing system

In this section, we introduce and showcase CREW – **C**orral+Whisk: **R**apid **E**lastic data processing with corral and Open**W**hisk – A new serverless data processing system, combining both platform and application re-designs to resolve the most predominant tensions of data processing performance for ad-hoc tasks, following closely the reference architecture defined. For that, we first briefly introduce CREW, highlighting different re-design options we utilized to address the tensions. Then, we present the execution and processing steps that enable it to improve on state-of-the-art systems. We conclude this section by reviewing current shortcomings and open issues of CREW before presenting a comparative evaluation in the next section. However, we want to highlight that CREW is not aiming to replace existing, well-adopted solutions such as Lithops or Wukong but to demonstrate the potential of applying application platform co-design to serverless data processing.

3.1. Addressing tensions through re-designs

Based on the reference architecture (see Fig. 1) and existing tensions in serverless data processing (see Section 2.3), we present a series of re-design options that address them through the augmentation of serverless data processing applications and serverless platforms. The presented re-designs are an excerpt of re-designs performed in the last few years by the authors and several students working on serverless data processing [47]. All re-designs were performed on the open-source serverless platform OpenWhisk [48] and the serverless data processing framework Corral [34]. For each of the following experimental re-designs, we studied several implementation options and evaluated the impact of using the re-designed systems for typical serverless workloads and data processing workloads, carefully selecting the most promising designs to present.

We note that the selection of Corral as a starting point was mainly motivated by the excellent performance we observed in prior experiments [19]. Moreover, the existing architecture of Corral was elegant and simple, allowing us to quickly prototype augmentations. While evaluating all solutions presented in Table 1, we found that many were not actively maintained or had complex and historically grown architectures that would have made fast prototyping more difficult.[3] Similarly, we selected OpenWhisk, as it was one of the few available and, at the time, actively maintained open-source serverless platforms

[2] A different usage as than serving web requests, which may lead to increased and even unforeseen overhead on the platform side.

[3] Admittedly, CREW now also is one of those complex systems that would be difficult to augment.

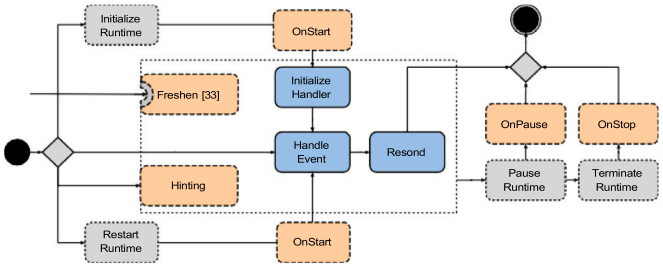

Fig. 2. Overview of FaaS platform life-cycle (gray) and added life-cycle-hooks (orange) and typical function execution (blue).

that exhibited most properties of a serverless system. While other options exist, many use Kubernetes as their sole orchestrator, while OpenWhisk also supports direct runc environments and custom orchestration, image loading, and caching. Thus, it functions as a good proxy compared to AWS Lambdas backend architecture [49], which gives us the confidence that the proposed platform re-designs could also be used in commercial settings.

Batching. Addressing **T4** through event batching yielded one of the most significant improvements by offering serverless data processing as a better means to send events. While serverless computation systems are often designed for web-facing tasks, they rarely offer the means to push correlated events at once. However, serverless data processing must simultaneously push hundreds or thousands of events to start massively parallel processing steps. Thus causing network overhead for spawning and observing executions. A common solution for such overheads is the batching of events into fewer messages to reduce traffic. However, this can hide the complexity of workloads from the platform in case these batches are only processed on the function level and also may require additional workarounds like bootstrapping [13].

After evaluating several options, we selected a platform-re-design, where we modified the OpenWhisk Invocation-API. Here, we introduced the option to send multiple invocation events in a new batch-event message. It is still up to the developer to choose the size of each batch and the time interval between sending batches, and the platform can still impose rate-limiting and throttling logic as before. The complexity of adding this feature in OpenWhisk is relatively low, as we only need to modify a single component in the controller.

Lifecyclehooks. The nature of serverless systems implies that resources are often only provisioned in response to events. Thus, the cold-start of resources has been identified as one of the critical issues for serverless applications [37]. Furthermore, resources only execute code during the processing of events, thus requiring that all computations such as code management, monitoring, and logging can only happen during the response to events [50]. Due to this design, most serverless applications perform computations that do not directly contribute to a response, impacting cost and execution times (**T7, T5**). Based on the observation that some of these tasks, e.g., connecting to a database or logging, do not need to happen for every event or only need to happen for very special life cycle events, e.g., starting a runtime, we implemented an extension to the serverless execution model that allows developers to react to events outside the normal event-response cycle, expanding on the ideas of Hunhoff et al. [51]. Specifically, we define additional, developer-defined code execution points that run when a runtime is starting, paused, stopped, or responding to events (see Fig. 2).

While generally beneficial, this change enables data processing applications to preload data on runtime creation or write data after job completion instead of doing so during every event. Moreover, we could also allow developers to react to execution failures, e.g., offering the developer strategies to influence the re-execution of the task. This simple yet versatile modification of the serverless programming model offers many opportunities to the developer to move code away from the critical execution path of a serverless data processing application. However, overly long-running life-cycle hooks can increase cold starts and also reserve resources of runtime hosts longer.

Function hinting. Serverless data processing also creates unique demands on computing platforms (**T4, T6**), specifically, when it comes to the volatile number of invocations that are needed to perform processing jobs. Here, cold starts due to an unexpected increase in invocations, which can quickly lead to long waiting times. However, the number of invocations and computing demand can be predicted relatively accurately during the execution of each processing step. Thus, we implemented hinting, which allows the driver to communicate associations between running functions and future invocations, which can be used to speed up sequences of functions. Knowing which runtimes are needed next and in what relative volume, a scheduler can extend the pre-warming concept that many platforms use without user intervention.

Here, we combined part of the life cycle hook implementation and allowed drivers to send upcoming execution hints to the platform, including anticipated duration per event. The platform would then trigger the freshen life-cycle-hook to perform a function warm-up or startup, depending on the state of the requested runtimes. For this, we expanded the OpenWhisk controller and the OpenWhisk controller to be able to prepare runtimes depending on received hints. Naturally, a production system should select careful limits on the type and size of hints to balance the ability to remain highly elastic and utilize all resources most effectively.

Automatic re-configuration. We discussed the uniform, one-size-fits-all model of current serverless (FaaS) deployments (**T2, T3**). An application/framework developer has only a limited set of options to address the sizing problem [39]. However, prior work in using samples of runtime execution for different configuration variations [39] showed promising results in quickly finding suitable runtime configurations even for large function compositions, such as they are typical in serverless data processing. Here, the sizing problem can be even more drastic for very skewed data, where, depending on the data, some functions may be lacking resources while others are done far too quickly and cause higher costs than would be needed.

Thus, we expanded the approach presented in CAT [39] by allowing the sizing of not only compositions but also the sizing of the same functions based on input mapping by implementing automatic re-configuration. This strategy utilizes two modifications of typical serverless data processing systems: (1) we deploy each function for the same step in a processing job multiple times to be able to manage skewed data by only sizing functions receiving more demanding inputs, and (2) we automate the observation of sizing data to quickly select appropriate sizes of all functions in a processing composition. The first part of this strategy is implemented on the driver side by dividing and deploying jobs across multiple deployments for each run. The second part of the strategy is implemented inside OpenWhisk. Here, we make use of the fact that a serverless platform, especially for data processing workloads, deploys many runtimes for the same function. Thus, we can size a portion of deployed runtimes differently to obtain the necessary samples for the CAT-Sizer [39]. This way, the serverless processing job quickly (within a few event executions) converges to a better-fitting configuration without developer interventions. In a production setting, a cloud provider may want to carefully select the number of sampling runtimes and also offer means to define the lower and upper bounds for runtimes to ensure that the sizeres' dynamic optimization causes no degradation in performance.

Feedbackapi. One platform-driven limitation of current serverless computing platforms is the way in which drivers can observe the execution functions. Essentially, a driver can either (1) perform synchronous invocations, which is infeasible at the typical scale of serverless data

processing, (2) use asynchronous invocations and observe the completion by polling the serverless platform until the completion is observed, (3) remove the need for observation by letting functions directly call other functions or lastly use platform features such as orchestrates, e.g., AWS StepFunctions. Letting functions call others directly can quickly lead to run-away computations or make debugging significantly more difficult [40]. Using the platform options may work in some cases, but it often is limited in the possible parallelism and can add significant costs. Thus, most existing frameworks use the polling method as it is the quickest and most effective option. However, polling also creates a lot of network overhead and stress on the serverless platform.

To address this issue, we implemented a Feedback API that changes how the completion of invocation is communicated to applications (**T1, T3**). Specifically, the feedback API sends a message from the serverless system for each completed invocation (or group of invocations) using a callback address. This way, a driver can listen to the callbacks instead of implementing any polling. Moreover, allowing to group completions based on event type enables the significant reduction of network traffic between driver and platform. In a production system, we may need to enable means to abuse this system. For the tested serverless data processing applications, we could observe significant improvements, especially in combination with event batching and Hinting.

Storageapi. Serverless data processing generates intermediate data that typically requires temporary storage and can be easily recreated. Since data objects are typically accessed by only a subset of functions, availability only needs to meet job requirements. The scalability of the storage system is crucial for effective data processing with parallel tasks. To balance the need for intermediate data and ingestion of raw data, many serverless data processing approaches utilize different data storage systems for raw and intermediate data [19]. However, finding a perfect match is challenging due to issues with data size, storage duration, and supported parallel reads (**T6-T8**).

To address this challenge, we implemented a transparent StorageAPI, allowing the automatic selection of storage services based on runtime needs. The system supports Redis, NTFS, S3, and DynamoDB, enabling the driver to switch between them based on observations, such as the number and size of intermediate results generated. Future improvements could involve alternative algorithms for more backends and fine-grained decisions. Even with the simple rules implemented, significant improvements in execution speed and throughput were observed.

Application aware function scheduling. In this design, we use the knowledge about the structure of a data processing application to aid task scheduling and pre-warming decisions made by a platform (**T3, T4**). Similar to the `Hinting`-Design, this strategy enables developers to communicate relationships between deployments and access patterns that can be used by a scheduler beforehand. We build this design based on a preliminary design by A. Fuerst [52], but instead of online predictions, we use two types of application information to aid the scheduling and especially the removal of runtimes in favor of others. In the first approach, we use information about the needed libraries within a deployment package to determine cold-start times. The assumption is that some libraries take significantly longer to load due to their size or functionality than others. Further, the presence of some libraries strongly correlates with initialization needs, such as connecting to databases before a runtime becomes truly operational [50]. We can also further include the use of starting and pausing life-cycle hooks in this equation, as these also indicate longer cold-start times. For the second approach, we focus on knowledge about the composition of a serverless application. By knowing how functions interact with each other, we can determine how likely a function will be called in the near future based on the last N observed invocations. With this, long-running complex compositions can ensure that runtimes that are needed later are not removed prematurely. Here, we combine the hinting approach discussed before to ensure that we always prioritize runtimes that will be needed in the near future while removing runtimes that are quick to restart and also not needed by any currently running application.

3.2. Implementation

Fig. 3 shows a high-level overview of `CREW` components, following the same reference architecture design presented in Fig. 1.

`CREW` is divided into four distinct parts, see Fig. 3. Firstly, the core driver component, which is an extension of the Corral-Project [34]. While the basis of this code already existed, the `CREW` version contains about four times as many lines of code and extends the original functionality significantly, making it a fully unique software artifact. It was used as a convenient starting point but mainly served to inform early prototyping efforts. Secondly, the modified version of OpenWhisk [48] also includes significant code changes to the original[4] in addition to significant changes to the runtime environments. Further, the Storage-API, a set of libraries and interfaces to interact with multiple cloud storage backends in the context of `CREW` without the need to know how to deploy and configure them, and the storage sidecar, a set of plugins that can dynamically be loaded and used by `CREW` to configure and deploy storage backends. In the following, we describe how each component is implemented, how the re-designs from the previous section are integrated into the respective components, and how the overall design enables ad-hoc serverless data processing.

As a first step, a data analyst needs to implement a workflow using the map-reduce pattern, typically implemented as a single Golang file containing both the map and reduce function and the main function that initializes `CREW` to run the job. The job itself is driven by two things: a config file, which contains all the information to deploy all the functions and access the storage backend, and tunable values to drive the job. In the following, we describe the steps `CREW` performs to execute such a job on the example of `TPC-H Query 6`, a simple query to count items in a *lineitem* relation in the TPC-H benchmark.

Once the main function starts, the following six steps are performed to perform the job:

Validation First, `CREW` checks the configuration to validate that all the necessary access rights are present. In the case of our exemplary query, this includes validating the Minio credentials, the OpenWhisk credentials, and the Kubernetes credentials.

Preparation Next, `CREW` queries the storage backend, evaluating based on the job definition which files are needed to start the job. This step informs the number of invocations needed to perform the job. Further, in this step, the **StorageAPI** feature is used to select an appropriate secondary storage backend. Using the meta-data information, we might also select parts of the requested data for preloading using the **OnStart-LifeCycleHook** feature. In the running example, that entails the discovery of the roughly 120 files needed to read the *lineitem* relation based on the total size. Here, a Redis database is selected with no preloading.

Deployment Once the appropriate strategy for the job is selected, `CREW` deploys all needed components for the job. If we use a secondary storage backend, we instruct the storage sidecar to deploy that backend. In parallel, `CREW` compiles an executable version for the targeted serverless platform. In either case, the user-provided go file source code is compiled into a binary that can run on the target platform, including preloading operations. During compilation, we also inject all needed configurations to access the storage backends and other resources of the serverless composition. Lastly, after the successful compilation, we might also use the **Hint** feature to prewarm the first set of runtimes before the job is started. Furthermore, `CREW` might deploy more than one compiled binary to enable **Automatic Reconfiguration** experiments during execution. In the example,

[4] $\sim 6\%$ additions in the codebase (excluding configurations and build files)

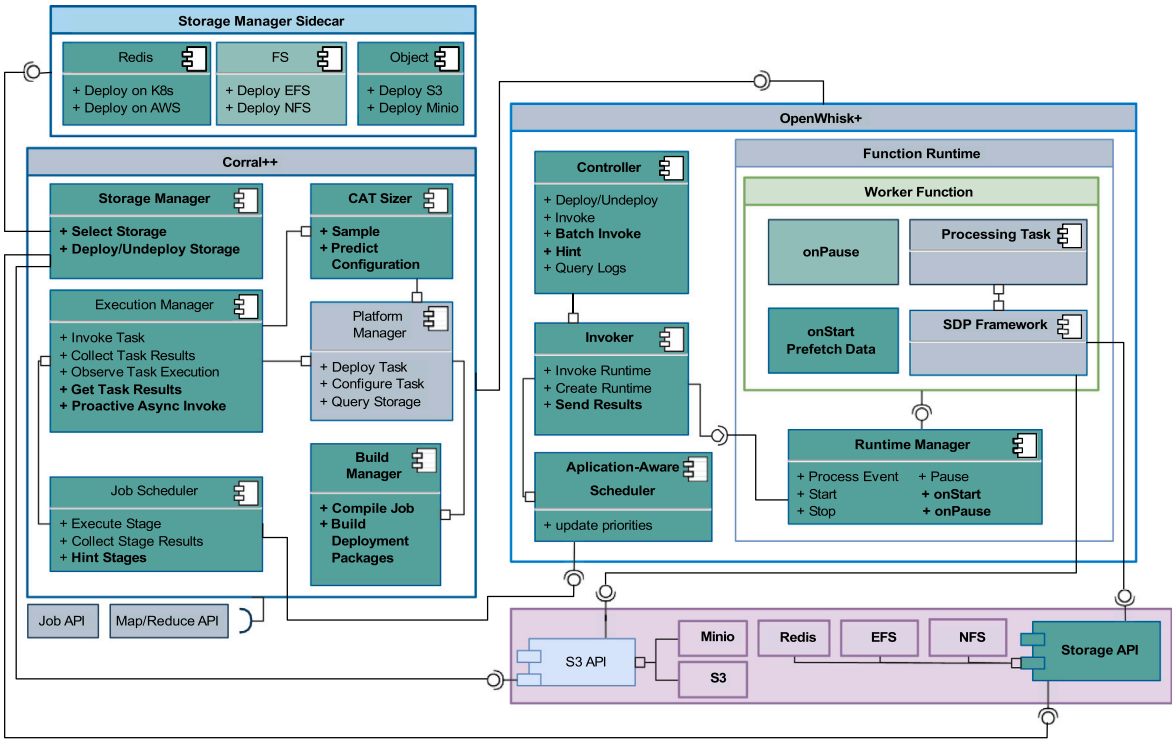

Fig. 3. High-level component overview of CREW, highlighting all integrated augmentations in comparison to the baseline (green, bold).

this includes the deployment of Redis in the Kubernetes cluster using a Helm client. Further, based on the average file size and capability reported by the OpenWhisk platform, we anticipate 128 parallel invocations. Accordingly, we use 128 as a hint.

Execution Finally, the job is executed in a stage-by-stage manner by subdividing all input files across several invocation events. The number of invocation events is determined by the used storage backed, the size of the input data, the maximum concurrency of the target platform, and the goal definition in the job config, e.g., if either speed or cost should be maximized. Once the number of invocations is determined, the invocations are sent to the target platform either using the **Batching** feature or using the Invocation-API of the platform. After performing all invocations in a stage, CREW observes the storage backend and serverless platform to wait for the completion of a stage. Once the stage is completed, the next stage is started. Before starting the next stage, CREW might trigger some **Re-configuration**, send **Hints** to the platform, or switch the storage backend to a different one based on the input data for the next stage.

In the case of `TPC-H Query 6`, we performed over 300 map invocations and a single reduce invocation. During the execution, CREW reconfigured the functions to use 512 MB of memory and reduced the number of parallel invocations to 64, as the throughput of the Minio backend was limited. Moreover, it utilized a single-server Redis instance to store the intermediate data.

Completion Once all stages are completed, CREW can download the final results from the storage backend or provide the analyst with a link if the results are too large to download. CREW can also be tasked to trigger a post-process step, e.g., invocating a different serverless application to generate a set of plots from the resulting data. In the running example, the final result is a single value immediately downloaded and displayed.

Termination Once the job is completed and all the results are collected, CREW terminates all deployed components if not otherwise specified in the config. Thus, the deployed database, deployed functions, and intermediate data are removed. In the case of the `TPC-H Query 6` example, we remove the Redis deployment and remove the deployed functions.

Each step might change based on the targeted platforms, supported platform features, and observations during the job execution. However, note that CREW is built to be self-contained. Thus, all resources needed to perform a processing job are created and removed for each run, embracing the ad-hoc nature of serverless data processing.

3.3. Limitations

CREW already offers performance, cost and scalability advantages for ad-hoc data processing. However, CREW uses a rather rudimentary map/reduce interface. While this is a very powerful primitive, novice developers will need ample time to understand it. A more accessible interface, e.g., a query language or a higher-level abstraction such as a SPARK API, could significantly improve the ease of use. However, using the existing primitives, a query language could easily be implemented.

Moreover, error recovery could be improved, in general, and is still an open issue in CREW; on the one hand, re-trying a single failed invocation will not significantly increase the cost or processing time of a job, but recovery of multiple errors will. In its current form, about 30% of CREW's codebase is dedicated to error recovery. With the increasing complexity of the serverless platform, this percentage will increase. Here, a more transparent way to deal with errors on the driver and platform side could significantly improve code quality and overall performance.

From the platform perspective, the current implementation still uses very little information on the global use of all functions. Consequently, both scheduling and load-balancing strategies could be improved to exploit locality and reduce throughput bottlenecks. Lastly, regarding error recovery, the current platform still offers little observability and

troubleshooting capabilities. We investigate the integration of tracing into OpenWhisk [40], which reveals that such a feature is easily integrated and will not significantly impact the system.

Overall, most of the platform modifications presented are not specific to OpenWhisk and could be applied to other serverless platforms, such as AWS Lambda. We can only assume, based on our observations on OpenWhisk, that both the platform impacts and application benefits would be similar; however, specific cost and resource models for other platforms might hinder adoption.

4. Experiment-driven evaluation

In the following, we show an evaluation of CREW using the TPC-H benchmark on a comparison of the original Corral+Openwhisk, herein referred to as Baseline, and a Spark-Operator [53] all using the same four node Kubernetes cluster and a connected high-performance minio cluster. CREW will behave as described in the previous sections and select these designs based on the job requirements. This evaluation aims to demonstrate the benefits of the application platform co-design approach in addressing performance and cost issues in serverless data processing. Hence, we compare the performance of CREW to the baseline to showcase the improvements that are possible and we compare CREW to Spark to show that serverless data processing can be competitive with more established tools on the same resources, thus, closing the gap between serverless and established tools while gaining the benefits of the serverless operation model. Here, we must point out that Spark is a far more mature tool than CREW and has been optimized for many years, but Spark-Operators is a fairly recent development and is far closer to the serverless data processing model than the original Spark implementation.

In this evaluation, we focus on the execution time, throughput and cost of the queries. When an organization performs queries such as in this evaluation on a regular and predictable basis, a dedicated cluster will be more cost-effective. However, for ad-hoc queries, where resources are only needed for a short time, and for less technical data analysis, serverless data processing is far more attractive [18,19]. Thus, for all following experiments, we always tested the performance from a fully cold cluster to target ad-hoc use.

4.1. Workload & protocol

For the evaluation, we use the TPC-H benchmark [54] as a workload. TPC-H is a decision support benchmark that measures the performance of ad-hoc queries on large data sets, originally designed for relational databases. However, the benchmark is also used to evaluate other data processing systems, such as Apache Spark [53] and is also used in the serverless domain to evaluate serverless data processing systems [19,55]. For the Baseline and CREW, we implemented the TPC-H queries in the map-reduce-paradigm, and for the Spark-Operator, we utilized the Spark-SQL library. For the evaluation, we utilize a subset of TPC-H queries (see Table B.3) with scaling factors 1 and 10. We limit the scaling factor to 10 due to the bandwidth within the experiment environment; otherwise, it can become a limiting factor, especially for the baseline. We repeat each query five times without reusing any deployed resources other than the Minio storage in between runs. The order of query execution is randomized to remove any bias.

For each query, the tools had to deploy all workers in an empty Kubernetes cluster as we were interested in the ad-hoc performance. For the Spark-Operator, we must manually size, scale, and configure the operator to fit the workload. However, the Spark-Operator does not allow using fractional vCPU in Kubernetes. Thus, the available CPU resources to the Spark Operator baseline were often higher than both the Baseline and CREW experiments. Towards that end, we sized the operator to match as close as possible to the maximum resource used by CREW for the same treatment. We also calculated the cost for executing each query by assuming the AWS Lambda pricing model for each runtime instance and fixed hourly cost for the Redis/Minio instances.

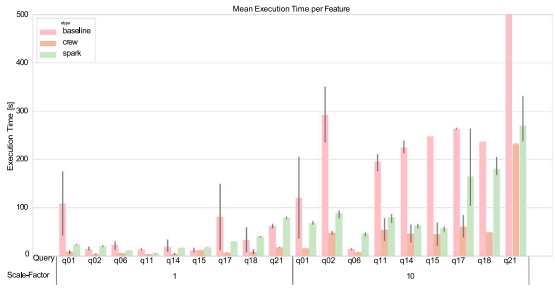

Fig. 4. Aggregated mean job execution time for the TPC-H application workload using Spark-Operator, CREW and the baseline for scale factors 1 and 10.

4.2. Results

In the flowing, we first present the results of the TPC-H queries for the selected scaling factors in terms of execution latency and throughput in Fig. 4 and Fig. 6 respectively. We then present selected results in detail, showcasing some of the differences introduced by using the different re-designs in CREW. Lastly, we present a summary table typical for the TPC-H benchmark as an extension to work presented in Werner et al. [19] in Table 2. In all results, we compare the baseline, an unmodified version of OpenWhisk and Corral, CREW and Spark, all using the same workload and hardware.

Firstly, in Fig. 4, we observe that the baseline is well suited for simple queries and able to outperform the Spark-Operator in some runs, thus confirming the results of Werner et al. [19]. However, for larger data sets, the baseline performance significantly decreases, indicating that the baseline implementation suffers from a set of inefficiencies. Here, CREW improves the performance over the baseline significantly and, in most runs, even outperforms the Spark Operator. For all but the heavy broadcast queries, CREW and Spark performed similarly to each other, indicating that the inter-worker communication in Spark might be a bottleneck in the Experimental Environment[5] deployment.

We examined these observations in more detail in Fig. 5. In the zoomed latency figure, we can see that CREW can start executions faster than both the baseline and Spark and distribute more work across available workers. For the baseline, we can see clear steps in the execution, a sign that the worker idles either due to IO operations or slow orchestration. The spark operator uses roughly the same amount of worker[6] through the execution. Although in Q1, we can see a delay between the first stage in the job and the execution across all worker nodes, this delay only happens in some runs and is unrelated to cold-start effects, as we ensured a warm execution environment for each experiment. We saw such delays of 1–20 s in about 10% of the runs, significantly impacting the overall performance of the Spark approach. For CREW, the executions started fast and remained high, and gaps appeared for write-heavy operations, where the storage backend could not cope with incoming requests. However, these gaps are overall relatively small (between 0.5–2 s). We assume that deployments on a hyperscaler, e.g., AWS, would not suffer similar issues, other than that CREW improves significantly over the baseline.

Looking into the contributing factors for this improvement, we firstly see a significantly higher throughput for CREW over the baseline, see Fig. 6. Note here that the maximum throughput for the used environment is about 10 Gbps. Since we calculated the throughput based on total data read/written during the execution time, we can see that CREW is able to operate at about half of this maximum capacity

[5] See Appendix B

[6] For Spark, we considered each thread as a separate worker, even if multiple workers run in parallel on the same deployed worker node.

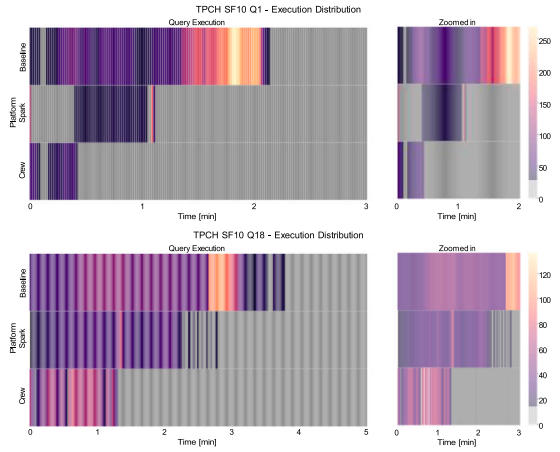

Fig. 5. In-depth execution distribution view of `TPC-H Query 1` and `TPC-H Query 18` for the TPC-H application workload for scale factor 10 for a single run, showing initialization gaps and orchestration/scheduling delays. Note how the baseline performs most work at the end due to stragglers in prior stages and how Spark needed time to initialize the workers.

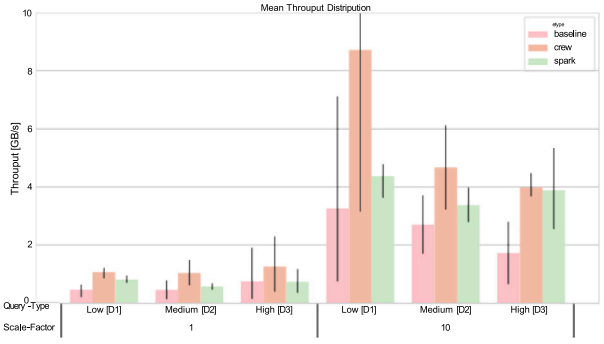

Fig. 6. Throughput overview for the TPC-H application workload using Spark-Operator, `CREW` and the serverless OpenWhisk baseline for scale factors 1 and 10, aggregated for Low, Medium, and High IO-intensive queries.

and manages to max out this capacity for simple queries with little random read/write access. For longer queries, the communication traffic within Kubernetes and OpenWhisk reduces the available bandwidth. For broadcast, heavy queries with lots of read and write operations (e.g., `TPC-H Query 21`), the Spark-Operator is able to operate at a higher throughput than the other two approaches, as Spark can also exploit node locality to share data, skipping the network, thus, having overall improved read/write throughput.

However, due to the fixed worker size and available resources, Spark is not necessarily able to outperform `CREW` even with the higher throughput, as some tasks benefit from higher parallelism, contributing to the overall lower execution time of `CREW`. This higher parallelism can also be observed in Fig. 7. Here, we can see that `CREW` used almost the same number of tasks as Spark, but while in Spark, tasks shared a fixed number of workers, in `CREW`, each task runs in a different function, allowing for higher total parallelism. This performance difference is also due to the more flexible worker (function) allocation in `CREW` that can spawn a large number of small works at the beginning of queries and then uses fewer high-memory functions in later stages. The higher memory consumption of `CREW` in Table 2 for the smaller scaling factor also indicates this. However, we point out that the analyst or operator of both Spark and `CREW` can always configure the maximum memory that should be allocated by setting the maximum number of workers/functions and the memory per worker.

We summarize the performance of Spark, `CREW`, and the baseline in Table 2. Here, we can first see that all three platforms are sensitive

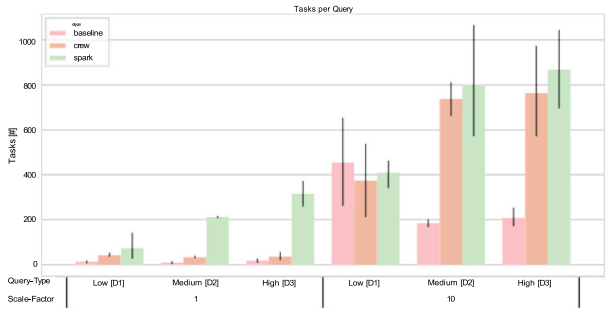

Fig. 7. Runtime usage for the TPC-H application workload using Spark-Operator, `CREW` and the serverless OpenWhisk baseline for scale factors 1 and 10, aggregated for Low, Medium, and High IO-intensive queries, see Table B.3.

Table 2
Main TPC-H application workload results for Spark-Operator, `CREW` and OpenWhisk (Baseline) for scale factors 1 and 10, expressing theoretical queries per hour (QphH), cost per 1000 queries per Hour (cost/kQphH) and used resources for each scale factor.

Platform	Scale-factor	QphH	cost/kQphH [$]*	Peak memory allocation[GB]	vCPU	IO [TB/h]
Baseline	1	159	0.08	3	1	1.51
Crew	1	992	2.50	15	1	5.56
Spark	1	133	0.68	6	2	3.07
Baseline	10	26	3.70	6	3	4.61
Crew	10	130	6.51	32	2	16.30
Spark	10	32	5.51	32	15	14.33

to the data volume (SF). This, however, is more an artifact of the used *Exp-Env.*, as we could not scale the network to the storage backend.

Nevertheless, `CREW` can almost perform 1000 queries per hour for SF = 1 and 130 for SF = 10, indicating that a better-scaled storage environment could archive similar performance for SF = 10 as for SF = 1. We also see that `CREW` is the most expensive,[7] primarily due to the extra costs of running a Redis instance. Note that at the same time, `CREW` never used more than two full vCPUs as each function never uses more than 40mili vCPUs for most operations, while Spark needed one vCPU per worker at a minimum + 1 vCPU for the Spark master. Thus, for On-Prem deployments, where vCPUs might be limited, using an approach such as `CREW` might be more beneficial as fewer CPU resources are committed. On the other hand, `CREW` will consume available memory more aggressively, as evident by the high memory usage for SF = 1. At the same time, Spark is very predictable as the works need to be preconfigured. However, we argue that the configuration limits still allow control of maximum memory consumption while utilizing the available resources more effectively.

4.3. Analysis

The results of the evaluation show that serverless systems can be similarly performant as far more advanced platforms such as Apache Spark. The high scalability and flexible worker deployment of serverless data processing approaches, in combination with high-performance storage options available in cloud environments, makes platforms like `CREW` competitive. Some of the optimizations that Spark developed over time, e.g., sorting and partitioning strategies, could also be applied to `CREW`, likely improving the performance further.

Concerning development, Spark is still far simpler, offering developers many abstraction libraries that let them use SQL or simple domain-specific languages (DSL) to define their data processing pipelines. However, deploying and operating Spark clusters, even using the simplification of Kubernetes, can quickly remove these advantages.

[7] We calculated the prices using the AWS Lambda for all function executions and prices for m6a.4xlarge instances for the Minio/Redis instances as well as for the spark worker instances. Based on prices from April 2022.

A developer must pick the precise libraries, operator, and worker images to get a Spark operator to work, leading to much complexity. Using any additional libraries, such as the S3 connector, forced us to build a custom version of the spark operator and worker images.

Further, we observed that the deployment, while eased, still required a lot of sizing-related tasks removed in the serverless data processing. Some of these issues are due to the high complexity of Sparks architecture, thus likely needing to be solved in the future. While approaches such as CREW can, over time, develop similar technical debt, the benefits of fully automated operational tasks will remain. With regards to the available programming abstractions, serverless data processing frameworks likely can, over time, adopt similar approaches as established tools such as Spark.

Based on a detailed analysis of individual queries, we can see that CREW now is constrained mainly by the available bandwidth and application behavior, e.g., how the queries are written and how data is shuffled and accessed. Thus, the serverless platform itself is not the limiting factor in the execution and performance of ad-hoc data processing. Instead, application logic and design now need improvements.

Overall, serverless data processing proved to be highly performant for ad-hoc data processing, especially in on-prem deployments. With the right storage backends, scale and performance can be increased to fit the needs of each use-case, especially in cloud settings. The design of CREW enables highly elastic scaling, fitting the needs of each processing step to the data while allowing data analysts to control cost by setting fixed budgets. All modifications in the serverless platform (OpenWhisk) did not affect the stability or performance of other applications (tenants). Some modifications, such as batching or feedback, introduce increased resource demand on the platform side. However, the reduced time applications run due to these changes and the reduction in polling requests compensate for these resource demands. Still, further improvements both to the application- and platform- side can further improve the performance.

5. Related work

Serverless computing is still an emerging field, with many opportunities and challenges [15]. Within the context of serverless, works by Jonas et al. [10] as well as Hellerstein et al. [56] and Castro et al. [57] present a foundational understanding of these opportunities and challenges. Indeed serverless data processing is especially affected by these challenges such as the need for "Serverless Ephemeral Storage and Serverless Durable Storage" [10], as well as the challenge that serverless functions "can only communicate through an autoscaling intermediary service" [56] but also provide specific application-platform re-designs that address these challenges in CREW through the *FeedbackAPI* and *StorageAPI*.

Serverless data processing is one of these opportunities, which find wide interest in research and industry [9,13,16,24,25,27,36,58–61]. Consequently, several excellent frameworks and tools to perform serverless data processing (See Table 1) have been proposed to enable serverless data processing. Many of them found very novel and needed workarounds to address existing challenges not only in storage or communication but also for the unique tensions that serverless platforms present. Here, novel solutions to tackle increased effects of cold-starts [13] or novel solutions of task distribution [24] already present very helpful solutions. With CREW, we benefited from these many findings and created yet another serverless compute framework; however, rather uniquely, we asked the question if some of the platform-driven limitations are necessary and should remain. Thus, resolving these workarounds by proving platform-level features to reduce cold-starts (*Hinting, LifeCycleHooks*) and task distribution (*Batching, FeedbackAPI*).

Additionally, many practitioners are looking into the limitations of existing serverless platforms [37,62–64] with the aim to improve and adapt applications to fit the serverless model. Hence, these limitations all reveal current tensions in serverless platforms that can similarly be addressed at the platform level. While we specifically focused on the task of serverless data processing and therefore addressed challenges for improved scheduling and deployment sizing in CREW for this use-case, we argue that the practice of application platform co-design can be used to address other existing limitations.

Moreover, several excellent approaches exist to deal with limitations in serverless data processing applications [65–67]. With CREW we strongly benefited from these approaches to further reduce the operational tasks a data analyst is faced when using cloud-based models. Indeed, we foresee that these approaches can and should also be integrated through application platform co-design [1] into both serverless platforms not only for the benefit of serverless data processing workloads but for serverless applications in general.

6. Conclusion

Platform-driven limitations and application workloads in serverless data processing create significant tensions, resulting in performance and cost impacts. This paper presents an initial review of these tensions and proposes a set of re-design options to mitigate the cost and performance issues following the application platform co-design approach [47]. By expanding engineering efforts to the serverless platform, we effectively address these tensions and introduce CREW, an innovative serverless data processing framework and computing platform tailored for ad-hoc processing [1]. Our evaluation using the TPC-H benchmark demonstrates that CREW outperforms conventional serverless data processing systems and performs comparably to Spark Operators on the same resources, validating the approach to tackle arising cloud computing complexity by also looking at the platform side.

Serverless data processing offers comparable performance to established methods while significantly reducing operational tasks for developers and data analysts. Therefore, it should be considered a viable alternative for big data processing. To meet the emerging application demand, platforms should adapt and re-design their limitations accordingly. Although our focus in this paper was on ad-hoc query processing, other trends in serverless computation, such as hardware resource abstraction, may enable serverless systems to tackle different processing tasks like machine learning model training and hyperparameter optimization [42].

Similar to the evolution of database management systems (DBMS) decades ago [68], we anticipate the divergence of commercial serverless platforms from the one-size-fits-all model. This will give rise to new platform variants that cater to specific application requirements. Moreover, as more fields require distributed, scalable, cost-effective, and on-demand solutions, the trend of delegating operational, management, and execution tasks to platforms will continue to grow. Consequently, there will be a need for diversified serverless platforms to accommodate these diverse application needs. Therefore, the development of new engineering methods to create such diversified application platforms becomes a crucial next step.

CRediT authorship contribution statement

Sebastian Werner: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Stefan Tai:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Sebastian Werner reports financial support, article publishing charges, and travel were provided by TU Berlin University.

Data availability

Links to the code and data are part of the manuscript.

Acknowledgments

Funded by the European Union (TEADAL, 101070186). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

Appendix A. Systematic literature review

With the aim of extracting a reference architecture for serverless data processing, we performed a systematic literature review (SLR) of serverless data processing and serverless platforms. While many systematic literature reviews for serverless computing already exists [16,58,69–72] none focuses on serverless data processing.

For the review, we *excluded publications published before 2014*, as serverless computing was first publicized by the release of AWS Lambda in 2014 [10], only *consider English peer-reviewed publications* that were *at least two pages in length*, exclude *all works published outside of scientific journals or conferences*, and we *exclude all secondary studies*. However, since serverless computing research remains an emerging field, we do make *an exception for pre-prints* if these were already accepted at a conference or journal or by authors that published other peer-reviewed work before.

In total, the searches yielded 484 candidates. After de-duplication and filtering, we had a total of 323 candidates. From these, we identified a total of 92 as relevant for extraction. Within this search, we found 22 general-purpose serverless data processing frameworks and a total of 61 data processing applications. While some of these applications represent either one-time implementations or implementations of sub-problems, others are built to solve generalized data processing queries, map-reduce jobs, or generalized encoding/transcoding jobs. Further, we identified four main use cases, ranging from typical data analytics tasks such as batch and stream processing, machine learning tasks, data encoding/transcoding tasks, and operations automation, mostly built on AWS [10]. Table A.2 shows a list of highly relevant publications identified by the search. These publications can be separated into four groups of contributions in the field of serverless data processing: (1) frameworks that enable developers to utilize serverless systems, (2) use-case implementations, (3) assessment methods, and (4) contributions proposing new features to address platform problems. In terms of use cases for serverless data processing, we identified a total of *121*, that can be broadly categorized into three groups, with some works presenting more than one. The majority of use-cases target: (1) batch and stream data processing tasks, consisting of scientific analysis or exploratory data analysis using map-reduce jobs, followed by (2) data encoding/transcoding tasks, such as video compression, and recently (3) machine learning tasks, such as inference workloads and even some model training.

A more detailed executed literature study can be found in [47] and by reviewing the raw data in the repository.[8]

[8] https://docs.google.com/spreadsheets/d/1ZIP712dKKmDOnd4oZ77pMg2YiVfn7-P2Q9ESEddTt0U

Table A.2
Excerpt of the search results depicting the most relevant findings.

Mete-data	Year	Target platform					Approach
References		AWS	GCF	MAF	ICF/Whisk	Other	
Jonas et al. [9]	2017	✓					Framework
Kim et al. [22]	2018	✓					Framework
Sampe et al. [13]	2018				✓		Framework
Klimovic et al. [73]	2018	✓					Feature
Lopex et al. [74]	2019			✓	✓		Framework
Fouladi et al. [12]	2019	✓					Framework
Pons et al. [20]	2019	✓	✓		✓		Feature
Perez et al. [75]	2019	✓					Feature
Carver et al. [24]	2019	✓					Framework
Perez et al. [76]	2019	✓					Framework
Gimenez-Alventos et al. [29]	2019					✓	Assessment
Müller et al. [14]	2020	✓					Framework
Perron et al. [26]	2020	✓					Framework
Goli et al. [77]	2020	✓					Assessment
Sanchez et al. [78]	2020		✓				Feature
Daw et al. [79]	2020				✓		Feature
Jain et al. [80]	2020					✓	Framework
Werner et al. [19]	2020	✓					Assessment
Jarachanthan et al. [81]	2021	✓					Framework
Sampe et al. [35]	2021	✓	✓	✓	✓		Framework

Table B.3
Used TPC-H queries, grouped by data volume and complexity.

	Tables	Joins	Groups	Unions	In-volume	IO	Out-volume	Label
Q01	1		1		Mid	Low		
Q06	1				Mid	Low	constant	D1
Q14	2	1			Mid	Low		
Q15	1		1		Mid	Low	small, sparse	D2
Q18	3	3		1	Mid	High		
Q02	5	4	1		Low	Mid	scaling	D3
Q11	3	2	1		Low	Mid		
Q17	2	1		1	High	Mid		
Q21	4	5	1		High	High		

Appendix B. Experiment setup

For the evaluation of CREW, we use the following experiment environment and adhere to the recommendations of cloud service benchmarking [82] for measurement and evaluation practices. For evaluating OpenWhisk-based deployments, we use a Kubernetes-Cluster consisting of four worker nodes, a single master, and additional VMs to deploy auxiliary services in the same 10 Gbit network. We use Kubernetes in version 1.21.0 and base all changes to OpenWhisk on the commit d741c87. For the Spark workload, we used Spark-Operator version `v1beta2-1.3.7-3.1.1`.

Specifically, we used an on-premises experiment environment (*Exp-Env.*) consisting of 6 machines; a detailed overview of the setup is depicted in Table B.4. For reproduction, any similarity-sized Kubernetes cluster should be sufficient. However, it is essential to provide at least a capacity for 128 parallel function runtimes[9] in OpenWhisk and at least 32 GB memory for the Minio deployment co-located in the same 10 Gbit network. Note that we needed to tune Minio to handle 10Gbit/s traffic for a sustained request traffic.[10]

We select a modified version of the decision support benchmark from the Transaction Processing Council for Ad-hoc/decision support (TPC-H) [83]. The ad-hoc nature of the queries fits well with the identified workloads, including mostly semi-structured data format seen as workloads during the literature review. The benchmark is well established with public comparable performance data on many different processing systems. Hence, this allows us to compare the

[9] Each runtime needs, on average, 512 MB of memory. Thus, around 64 GB of free runtime memory after deploying OpenWhisk.

[10] We observed that setting Minio to allow up to 100000 requests instead of the memory-based configuration works well in our experiments.

Table B.4
Cluster overview of *Exp-Env.* connected by a 10 Gb/s network.

Name	CPU Model	Cores	Mem [GB]
Client	Xeon E3-1270 v6	4	64
KMaster	Xeon E5–2690	8	32
KNode1	Xeon E5–2690	8	32
Knode2	2x Xeon E5–2430	12	32
Knode4	Xeon Silver 4114	10	16
Knode5	Xeon Silver 4114	10	16
Minio	Xeon Silver 4114	10	48

performance of different serverless implementations with comparable processing systems such as Apache Spark. While the benchmark is targeted for Database systems, it has been used throughout research and industry to evaluate other data processing systems [19,84,85,85,86]. However, as the benchmark is originally designed to evaluate database systems, it also includes some database-specific workload elements, such as concurrent updates and modifications that are supposed to run in parallel to the processing workload to test the ability of a database to handle updates to the raw data as well as transaction management. However, these operations would mostly test the storage systems we use (Minio) instead of the serverless compute platforms and processing frameworks we are evaluating. Thus, we omit these concurrent updates from the benchmark, which is common in evaluating the performance of processing systems [19,84,85,85,86]. We implement all queries listed in Table B.3 using `CREW` in a program called corral-tpch.[11] The implementation is largely compatible with the original version of Corral and the modified version presented, thus allowing for a comparison between an unmodified Corral version and `CREW`. We can trigger different design features and other `CREW` configurations through a config file.

References

[1] S. Werner, S. Tai, Application-Platform Co-design for Serverless Data Processing, in: H. Hacid, O. Kao, M. Mecella, N. Moha, H.y. Paik (Eds.), Service-Oriented Computing, Springer International Publishing, Cham, 2021, pp. 627–640.

[2] M. Fragkoulis, P. Carbone, V. Kalavri, A. Katsifodimos, A survey on the evolution of stream processing systems, 2020, arXiv:2008.00842.

[3] H.L. Berghel, Simplified integration of prolog with rdbms, SIGMIS Database 16 (1985) 3–12, http://dx.doi.org/10.1145/2147769.2147770.

[4] A. Abouzeid, K. Bajda-Pawlikowski, D. Abadi, A. Silberschatz, A. Rasin, Hadoopdb: An architectural hybrid of mapreduce and dbms technologies for analytical workloads, Proc. VLDB Endow 2 (2009) 922–933, http://dx.doi.org/10.14778/1687627.1687731.

[5] M. Hahmann, C. Hartmann, L. Kegel, D. Habich, W. Lehner, Big by blocks: Modular analytics, it Inf. Technol. 58 (2016) 176–185, http://dx.doi.org/10.1515/itit-2016-0003.

[6] Apache Software Foundation, Apache spark, 2014, http://spark.apache.org/. (Online; Accessed 21 December 2022).

[7] M. Zaharia, M. Chowdhury, M.J. Franklin, S. Shenker, I. Stoica, Spark: Cluster computing with working sets, in: 2nd USENIX Workshop on Hot Topics in Cloud Computing, HotCloud 10, 2010.

[8] Amazon Web Services, Inc., AWS emr, 2009, https://aws.amazon.com/emr/. (Online; Accessed 21 December 2022).

[9] E. Jonas, Q. Pu, S. Venkataraman, I. Stoica, B. Recht, Occupy the cloud: Distributed computing for the 99 percent, in: Proceedings of the 2017 Symposium on Cloud Computing, ACM, New York, NY, USA, 2017, pp. 445–451, http://dx.doi.org/10.1145/3127479.3128601.

[10] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J.E. Gonzales, R.A. Popa, I. Stoica, D.A. Patterson, Cloud programming simplified: A berkeley view on serverless computing, 2019, URL: http://arxiv.org/abs/1812.03651.

[11] G.C. Fox, V. Ishakian, V. Muthusamy, A. Slominski, Status of serverless computing and function-as-a-service(faas) in industry and research, 2017, CoRR abs/1708.08028. URL: http://arxiv.org/abs/1708.08028.

[12] S. Fouladi, F. Romero, D. Iter, Q. Li, S. Chatterjee, C. Kozyrakis, M. Zaharia, K. Winstein, From laptop to lambda: Outsourcing everyday jobs to thousands of transient functional containers, in: 2019 USENIX Annual Technical Conference, USENIX, Renton, WA, USA, 2019, pp. 475–488, URL: https://www.usenix.org/conference/atc19/presentation/fouladi.

[13] G. Sampé, P. Sánchez-Artigas, Serverless data analytics in the IBM cloud, in: Proceedings of the 19th International Middleware Conference Industry, ACM, New York, NY, USA, 2018, pp. 1–8, http://dx.doi.org/10.1145/3284028.3284029.

[14] I. Müller, R. Marroquın, G. Alonso, Lambada: Interactive data analytics on cold data using serverless cloud infrastructure, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, ACM, New York, NY, USA, 2020, pp. 115–130, http://dx.doi.org/10.1145/3318464.3389758.

[15] J. Kuhlenkamp, S. Werner, S. Tai, The IFS and buts of less is more: A serverless computing reality check, in: 2020 IEEE International Conference on Cloud Engineering, IC2E, IEEE, 2020, pp. 154–161.

[16] P. Leitner, E. Wittern, J. Spillner, W. Hummer, A mixed-method empirical study of function-as-a-service software development in industrial practice, J. Syst. Softw. 149 (2018) 340–359, http://dx.doi.org/10.1016/j.jss.2018.12.013, URL: http://www.sciencedirect.com/science/article/pii/S0164121218302735.

[17] V. Markl, Mosaics in big data: Stratosphere, apache flink, and beyond, in: Proceedings of the 12th ACM International Conference on Distributed and Event-Based Systems, Association for Computing Machinery, New York, NY, USA, 2018, pp. 7–13, http://dx.doi.org/10.1145/3210284.3214344.

[18] S. Werner, J. Kuhlenkamp, M. Klems, S. Müller, Serverless big data processing using matrix multiplication as example, in: Proceedings of the IEEE International Conference on Big Data, IEEE, Seattle, WA, USA, 2018, pp. 358–365, http://dx.doi.org/10.1109/BigData.2018.8622362.

[19] S. Werner, R. Girke, J. Kuhlenkamp, An evaluation of serverless data processing frameworks, in: Proceedings of the 2020 Sixth International Workshop on Serverless Computing, ACM, New York, NY, USA, 2020, pp. 19–24, http://dx.doi.org/10.1145/3429880.3430095.

[20] D. Barcelona-Pons, M. Sánchez-Artigas, G. París, P. Sutra, P. García-López, On the faas track: Building stateful distributed applications with serverless architectures, in: Proceedings of the 20th International Middleware Conference, ACM, New York, NY, USA, 2019, pp. 41–54, http://dx.doi.org/10.1145/3361525.3361535.

[21] J. Klein, Reference Architectures for Big Data Systems, Carnegie Mellon University, Software Engineering Institute's Insights (blog), 2017, URL: https://insights.sei.cmu.edu/blog/reference-architectures-for-big-data-systems/. (Online; Accessed 23 June 2023).

[22] Y. Kim, J. Lin, Serverless data analytics with flint, in: 11th International Conference on Cloud Computing, IEEE, New York, NY, USA, 2018, pp. 451–455, http://dx.doi.org/10.1109/CLOUD.2018.00063.

[23] Q. Pu, S. Venkataraman, I. Stoica, Shuffling, fast and slow: scalable analytics on serverless infrastructure, in: 16th USENIX Symposium on Networked Systems Design and Implementation, USENIX Association, Boston, MA, USA, 2019, pp. 193–206, URL: https://www.usenix.org/conference/nsdi19/presentation/pu.

[24] B. Carver, J. Zhang, A. Wang, Y. Cheng, In search of a fast and efficient serverless dag engine, in: 2019 IEEE/ACM Fourth International Parallel Data Systems Workshop, PDSW, 2019, pp. 1–10, http://dx.doi.org/10.1109/PDSW49588.2019.00005.

[25] J. Sampe, P. Garcia-Lopez, M. Sanchez-Artigas, G. Vernik, P. Roca-Llaberia, A. Arjona, Toward multicloud access transparency in serverless computing, IEEE Softw. 38 (2021) 68–74, http://dx.doi.org/10.1109/MS.2020.3029994.

[26] M. Perron, R. Castro Fernandez, D. DeWitt, S. Madden, Starling: A scalable query engine on cloud functions, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 2020, pp. 131–141, http://dx.doi.org/10.1145/3318464.3380609.

[27] Qubole, Apache spark on AWS lambda, 2017, https://www.qubole.com/blog/spark-on-aws-lambda/. (Online; Accessed 21 December 2022).

[28] F. Oliveira, S. Suneja, S. Nadgowda, P. Nagpurkar, C. Isci, Opvis: Extensible, cross-platform operational visibility and analytics for cloud, in: Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference: Industrial Track, Association for Computing Machinery, New York, NY, USA, 2017, pp. 43–49, http://dx.doi.org/10.1145/3154448.3154455.

[29] V. Giménez-Alventosa, G. Moltó, M. Caballer, A framework and a performance assessment for serverless mapreduce on aws lambda, Future Gener. Comput. Syst. 97 (2019) 259–274, http://dx.doi.org/10.1016/j.future.2019.02.057, URL: https://www.sciencedirect.com/science/article/pii/S0167739X18325172.

[30] O. Nahyl, GitHub repository: Ooso, 2017, https://github.com/d2si-oss/ooso. (Online; Accessed 21 December 2022).

[31] J. Carreira, P. Fonseca, A. Tumanov, A. Zhang, R. Katz, A case for serverless machine learning, in: Workshop on Systems for ML and Open Source Software at NeurIPS, 2018.

[32] A. Pérez, G. Moltó, M. Caballer, A. Calatrava, Serverless computing for container-based architectures, Future Gener. Comput. Syst. 83 (2018) 50–59.

[33] C.K. Dehury, S.N. Srirama, T.R. Chhetri, Ccodamic: A framework for coherent coordination of data migration and computation platforms, Future Gener. Comput. Syst. 109 (2020) 1–16, http://dx.doi.org/10.1016/j.future.2020.03.029, URL: https://www.sciencedirect.com/science/article/pii/S0167739X19330924.

[11] https://github.com/ISE-SMILE/corral-tpc-h

[34] B. Congdon, GitHub repository: corral, 2018, http://github.com/bcongdon/corral. (Online; Accessed 26 February 2019).
[35] J. Sampe, M. Sanchez-Artigas, G. Vernik, I. Yehekzel, P. Garcia-Lopez, Outsourcing data processing jobs with lithops, IEEE Trans. Cloud Comput. (2021) 1, http://dx.doi.org/10.1109/TCC.2021.3129000.
[36] B. Congdon, Introducing Corral: A serverless MapReduce framework, 2018, http://benjamincongdon.me/blog/2018/05/02/Introducing-Corral-A-Serverless-MapReduce-Framework/. (Online; Accessed 26 February 2019).
[37] J. Manner, M. Endreß, T. Heckel, G. Wirtz, Cold start influencing factors in function as a service, in: Proceedings of the 3rd International Workshop on Serverless Computing, IEEE, New York, NY, USA, 2018, pp. 181–188, http://dx.doi.org/10.1109/UCC-Companion.2018.00054.
[38] S. Eismann, J. Grohmann, E van Eyk, N. Herbst, S. Kounev, Predicting the costs of serverless workflows, in: ACM/SPEC International Conference on Performance Engineering, ACM, New York, NY, USA, 2020, http://dx.doi.org/10.1145/3358960.3379133.
[39] J. Kuhlenkamp, S. Werner, C.H. Tran, S. Tai, Synthesizing configuration tactics for exercising hidden options in serverless systems, in: International Conference on Advanced Information Systems Engineering, Springer, 2022, pp. 36–44, http://dx.doi.org/10.1007/978-3-031-07481-3_5, URL: https://arxiv.org/abs/2205.15904.
[40] M.C. Borges, S. Werner, A. Kilic, FaaSter Troubleshooting - Evaluating Distributed Tracing Approaches for Serverless Applications, in: 2021 IEEE International Conference on Cloud Engineering, IC2E, IEEE, 2021.
[41] J. Kuhlenkamp, S. Werner, M.C. Borges, K. E. Tal, S. Tai, An evaluation of FAAS platforms as a foundation for serverless big data processing, in: Conference on Utility and Cloud Computing, ACM, New York, NY, USA, 2019, pp. 1–9, http://dx.doi.org/10.1145/3344341.3368796.
[42] S. Werner, T. Schirmer, Hardless: A Generalized Serverless Compute Architecture for Hardware Processing Accelerators, in: 2022 IEEE International Conference on Cloud Engineering, IC2E, 2022, pp. 79–84, http://dx.doi.org/10.1109/IC2E55432.2022.00016.
[43] D. Bermbach, A.S. Karakaya, S. Buchholz, Using application knowledge to reduce cold starts in faas services, in: 35th ACM/SIGAPP Symposium on Applied Computing, ACM, New York, NY, USA, 2020, http://dx.doi.org/10.1145/3341105.3373909.
[44] J. Kuhlenkamp, S. Werner, M.C. Borges, D. Ernst, All but One: Faas Platform Elasticity Revisited, SIGAPP Appl. Comput. Rev 20 (2020) 5–19, http://dx.doi.org/10.1145/3429204.3429205.
[45] T. Schirmer, J. Scheuner, T. Pfandzelter, D. Bermbach, Fusionize: Improving serverless application performance through feedback-driven function fusion, 2022, arXiv:2204.11533.
[46] J. Sampé, M. Sánchez-Artigas, P. García-López, G. París, Data-driven serverless functions for object storage, in: Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference, ACM, New York, NY, USA, 2017, pp. 121–133, http://dx.doi.org/10.1145/3135974.3135980.
[47] S. Werner, Serverless Data Processing: Application Platfrom Co-Design (Ph.D. thesis), TU Berlin, 2023.
[48] Apache Software Foundation, Apache openwhisk, 2018, http://openwhisk.incubator.apache.org. (Online; Accessed 15 October 2017).
[49] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, D.M. Popa, Firecracker: Lightweight virtualization for serverless applications, in: 17th USENIX Symposium on Networked Systems Design and Implementation, USENIX Association, Santa Clara, CA, 2020, pp. 419–434, URL: https://www.usenix.org/conference/nsdi20/presentation/agache.
[50] S. Werner, J. Kuhlenkamp, F. Pallas, N. Anders, N. Mucaj, O. Tsaplina, C. Schmidt, K. Yildirim, Diminuendo! Tactics in support of faas migrations, in: M. Paasivaara, P. Kruchten (Eds.), Agile Processes in Software Engineering and Extreme Programming – Workshops, Springer International Publishing, 2020, pp. 125–132, http://dx.doi.org/10.1007/978-3-030-58858-8_13.
[51] E. Hunhoff, S. Irshad, V. Thurimella, A. Tariq, E. Rozner, Proactive serverless function resource management, in: Proceedings of the 2020 Sixth International Workshop on Serverless Computing, Association for Computing Machinery, New York, NY, USA, 2020, pp. 61–66, http://dx.doi.org/10.1145/3429880.3430102.
[52] A. Fuerst, P. Sharma, Faascache: Keeping serverless computing alive with greedy-dual caching, in: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Association for Computing Machinery, New York, NY, USA, 2021, pp. 386–400, http://dx.doi.org/10.1145/3445814.3446757.
[53] A.S. Foundation, Apache spark on kubernetes, 2021, https://spark.apache.org/docs/latest/running-on-kubernetes.html. (Online; Accessed 21 December 2022).
[54] T.P.P. Counci, TPC benchmark H standard specification revision 3.0.1, 2022, https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf. (Online; Accessed 21 December 2022).
[55] J. Spillner, Transformation of python applications into function-as-a-service deployments, 2017, URL: http://arxiv.org/abs/1705.08169. unpublished.
[56] J.M. Hellerstein, J.M. Faleiro, J.E. Gonzalez, V. Schleier-Smith, A. Tumanov, C. Wu, Serverless computing: One step forward, two steps back, in: CIDR 2019, 9th Biennial Conference on Innovative Data Systems Research, CIDR, 2019, URL: http://cidrdb.org/cidr2019/papers/p119-hellerstein-cidr19.pdf.
[57] P. Castro, V. Ishakian, V. Muthusamy, A. Slominski, The rise of serverless computing, Commun. ACM 62 (2019) 44–54, http://dx.doi.org/10.1145/3368454.
[58] Kuhlenkamp J., S. Werner, Benchmarking FaaS Platforms: Call for Community Participation, in: Proceedings of the 3rd International Workshop on Serverless Computing, IEEE, Z¨rich, Switzerland, 2018, pp. 189–194, http://dx.doi.org/10.1109/UCC-Companion.2018.00055.
[59] V. Yussupov, U. Breitenbu¨cher, F. Leymann, M. Wurster, A systematic mapping study on engineering function-as-a-service platforms and tools, in: Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing, ACM, New York, NY, USA, 2019, pp. 229–240, http://dx.doi.org/10.1145/3344341.3368803.
[60] D. Taibi, N. E. Ioini, C. Pahl, J.R.S. Niederkofler, Patterns for serverless functions (function-as-a-service): A multivocal literature review, in: Proceedings of the 10th International Conference on Cloud Computing and Services Science, CLOSER, Research Gate. Preprint, 2020.
[61] J. Scheuner, P. Leitner, Function-as-a-service performance evaluation: A multivocal literature review, J. Syst. Softw. (2020) 110708, http://dx.doi.org/10.1016/j.jss.2020.110708, URL: http://www.sciencedirect.com/science/article/pii/S0164121220301527.
[62] M. Grambow, T. Pfandzelter, L. Burchard, C. Schubert, M. Zhao, D. Bermbach, Befaas: An application-centric benchmarking framework for faas platforms, in: 2021 IEEE International Conference on Cloud Engineering, IC2E, IEEE, 2021, pp. 1–8.
[63] D. Jackson, G. Clynch, An investigation of the impact of language runtime on the performance and cost of serverless functions, in: Proceedings of the 3rd International Workshop on Serverless Computing, IEEE, New York, NY, USA, 2018, pp. 154–160, http://dx.doi.org/10.1109/UCC-Companion.2018.00050, URL: http://ieeexplore.ieee.org/abstract/document/8605773.
[64] E. van Eyk, J. Scheuner, S. Eismann, C.L. Abad, A. Iosup, Beyond microbenchmarks: The spec-rg vision for a comprehensive serverless benchmark, in: 2020 ACM/SPEC International Conference on Performance Engineering, ACM, New York, NY, USA, 2020, pp. 197–208, http://dx.doi.org/10.1145/3375555.3384381.
[65] López P.G., M. Sánchez-Artigas, G. Parıs, D.B. Pons, A.R. Ollobarren, D.A. Pinto, Comparison of faas orchestration systems, in: Proceedings of the 3rd International Workshop on Serverless Computing, IEEE, New York, NY, USA, 2018, pp. 148–153, http://dx.doi.org/10.1109/UCC-Companion.2018.00049.
[66] V. Gupta, D. Carrano, Y. Yang, V. Shankar, T. Courtade, K. Ramchandran, Serverless straggler mitigation using local error-correcting codes, 2020, arXiv:2001.07490.
[67] S. Eismann, L. Bui, J. Grohmann, C.L. Abad, N. Herbst, S. Kounev, Sizeless: Predicting the optimal size of serverless functions, 2021, Preprint.
[68] M. Stonebraker, U. Cetintemel, "One size fits all": An idea whose time has come and gone, in: 21st International Conference on Data Engineering, ICDE'05, 2005, pp. 2–11, http://dx.doi.org/10.1109/ICDE.2005.1.
[69] S. Eismann, J. Scheuner, E. Van Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. Abad, A. Iosup, The state of serverless applications: Collection, characterization, and community consensus, IEEE Trans. Softw. Eng. (2021) 1, http://dx.doi.org/10.1109/TSE.2021.3113940.
[70] H.B. Hassan, S.A. Barakat, Q.I. Sarhan, Survey on serverless computing, J. Cloud Comput. 10 (2021) 1–29.
[71] V. Yussupov, U. Breitenbücher, F. Leymann, M. Wurster, A systematic mapping study on engineering function-as-a-service platforms and tools, in: 12th IEEE/ACM International Conference on Utility and Cloud Computing, ACM, New York, NY, USA, 2019, pp. 229–240, http://dx.doi.org/10.1145/3344341.3368803.
[72] H. Shafiei, A. Khonsari, P. Mousavi, Serverless computing: A survey of opportunities, challenges and applications, 2019, arXiv preprint arXiv:1911.01296.
[73] A. Klimovic, Y. Wang, P. Stuedi, A. Trivedi, J. Pfefferle, C. Kozyrakis, Pocket: Elastic ephemeral storage for serverless analytics, in: 12th USENIX Conference on Operating Systems Design and Implementation, USENIX Association, Berkeley, CA, USA, 2018, pp. 427–444, URL: http://dl.acm.org/citation.cfm?id=3291168.3291200.
[74] P. Garćia-López, M. Sańchez-Artigas, S. Shillaker, P. Pietzuch, D. Breitgand, G. Vernik, P. Sutra, T. Tarrant, A.J. Ferrer, Servermix: Tradeoffs and challenges of serverless data analytics, 2019, arXiv:1907.11465.
[75] A. Pérez, G. Moltó, M. Caballer, A. Calatrava, A programming model and middleware for high throughput serverless computing applications, in: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, Association for Computing Machinery, New York, NY, USA, 2019, pp. 106–113, http://dx.doi.org/10.1145/3297280.3297292.
[76] A. Pérez, S. Risco, D.M. Naranjo, M. Caballer, G. Moltó, On-premises serverless computing for event-driven data processing applications, in: 2019 IEEE 12th International Conference on Cloud Computing, CLOUD, 2019, pp. 414–421, http://dx.doi.org/10.1109/CLOUD.2019.00073.
[77] A. Goli, O. Hajihassani, H. Khazaei, O. Ardakanian, M. Rashidi, T. Dauphinee, Migrating from monolithic to serverless: A fintech case study, in: Companion of the ACM/SPEC International Conference on Performance Engineering, ACM, New York, NY, USA, 2020, p. 2025, http://dx.doi.org/10.1145/3375555.3384380.

[78] G.T. Sánchez-Artigas M. Eizaguirre, G. Vernik, L. Stuart, P. García-López, Primula: A practical shuffle/sort operator for serverless computing, in: Proceedings of the 21st International Middleware Conference Industrial Track, Association for Computing Machinery, New York, NY, USA, 2020, pp. 31–37, http://dx.doi.org/10.1145/3429357.3430522.

[79] N. Daw, U. Bellur, P. Kulkarni, Xanadu: Mitigating cascading cold starts in serverless function chain deployments, in: Proceedings of the 21st International Middleware Conference, Association for Computing Machinery, New York, NY, USA, 2020, pp. 356–370, http://dx.doi.org/10.1145/3423211.3425690.

[80] A. Jain, A.F. Baarzi, G. Kesidis, B. Urgaonkar, N. Alfares, M. Kandemir, Splitserve: Efficiently splitting apache spark jobs across FAAS and IAAS, in: Proceedings of the 21st International Middleware Conference, Association for Computing Machinery, New York, NY, USA, 2020, pp. 236–250, http://dx.doi.org/10.1145/3423211.3425695.

[81] J. Jarachanthan, L. Chen, F. Xu, B. Li, Astra: Autonomous serverless analytics with cost-efficiency and QOS-awareness, in: 2021 IEEE International Parallel and Distributed Processing Symposium, IPDPS, 2021, pp. 756–765, http://dx.doi.org/10.1109/IPDPS49936.2021.00085.

[82] D. Bermbach, E. Wittern, S. Tai, Cloud Service Benchmarking: Measuring Quality of Cloud Services from a Client Perspective, Springer International Publishing, Cham, 2017.

[83] M. Poess, C. Floyd, New tpc benchmarks for decision support and web commerce, SIGMOD Rec 29 (2000) 64–71, http://dx.doi.org/10.1145/369275.369291.

[84] M. Wawrzoniak, R. Müller, G. Alonso, Boxer: Data analytics on network-enabled serverless platforms, in: 11th Annual Conference on Innovative Data Systems Research, CIDR 2021, 2021.

[85] D. Justen, Cost-efficiency and performance robustness in serverless data exchange, in: Proceedings of the 2022 International Conference on Management of Data, 2022, pp. 2506–2508.

[86] T. Bodner, T. Pietz, L.J. Bollmeier, D. Ritter, Doppler: Understanding serverless query execution, in: Proceedings of the International Workshop on Big Data in Emergent Distributed Environments, 2022, pp. 1–4.

Sebastian Werner is a senior researcher at the Information Systems Engineering chair at TU Berlin, Germany. He completed his Ph.D. in Computer Science in 2023 in the area of serverless technology and the design of distributed cloud-based applications. Sebastian has long experience working on European and national projects and is currently a work package leader in Horizon Europe Project TEADAL, focusing on accountability and trustworthiness in federated data lakes. His current research focuses on platform-based applications' sustainability, maintainability, and trustworthiness. Sebastian is passionate about addressing the challenges of the next generation of cloud-computing platforms in his research and teaching.

Stefan Tai is Full Professor and Head of Chair Information Systems Engineering at TU Berlin, Germany (2014-present). Prior to that, he was a Full Professor at the Karlsruhe Institute of Technology (2007–2014) and a Research Staff Member at the IBM Thomas J.Watson Research Center in New York, USA (1999–2007). He also held concurrent posts as Director of research labs and is a member of corporate supervisory and advisory boards. He earned his Ph.D. in Computer Science from TU Berlin in 1999. Stefan's research focuses on next-generation, platform-based distributed software systems that meet complex system qualities, providing for technology, social, and business innovation. Platforms of interest include cloud platforms and blockchain networks, and their meaningful combination and interplay.